

Medical Concierge — Developer Guide

An educational walkthrough for a new developer: what this application is, exactly how it works end to end, and how Docker is used to run it.

1. What you're looking at

Medical Concierge is a **single-user, self-hosted** web application for a patient juggling multiple doctors who don't coordinate with each other. The user photographs pill bottles, uploads visit-note PDFs, or snaps pictures of handwritten prescriptions. The app:

1. **Reads** those documents with a vision-capable LLM (Claude),
2. **Normalizes** every medicine/supplement name against RxNorm, the U.S. National Library of Medicine's drug vocabulary,
3. **Screens** the combined record set for drug–drug interactions, drug–supplement/vitamin/mineral interactions, duplicate therapy, and nutrient depletions,
4. **Scores its own confidence** at every step and flags anything shaky, and
5. **Exports** everything — including a clinician-readable PDF the patient can hand to a doctor.

Two framing principles shape almost every design decision in the codebase:

- **This is not a diagnostic device.** Every output is phrased as "worth discussing with your doctor/pharmacist," never as an instruction. You'll see this discipline in prompt text, UI copy, and PDF wording alike.
- **The LLM is never the final authority on a drug's identity.** The model reads documents; RxNorm decides what a name actually refers to. Anything that can't be confidently normalized is kept but flagged, never guessed.

Related documents: `ARCHITECTURE.md` (the long-term system plan this MVP fits into), `MVP_INGESTION_AGENTS.md` (why the ingestion design is what it is), `WINDOWS_SETUP.md` (end-user setup), `../deploy/README.md` (deployment).

2. The 10,000-foot view

```
Browser (static/index.html – one self-contained file)
|  fetch()
v
FastAPI app (app/main.py + app/api/routes.py)
|
|-- POST /api/ingest/{medicine|supplement}
|   |
|   v
|   agents/ -----> ingestion/file_loader.py    (PDF -> page images)
|   |               ingestion/multimodal_extractor.py (Claude vision,
|   |               forced tool call)
|   |-----> normalization/rxnorm_client.py    (RxNorm API)
|   |               normalization/supplement_terms.py (local fallback)
|   v
|   storage/store.py  (SQLite)
|
|-- GET /api/findings --> interactions/engine.py + knowledge_base.py
|                           (deterministic screening, no LLM, no network)
|
'-- GET /api/export ----> export/pdf_report.py (PyMuPDF) | CSV | JSON
```

Tech stack:

Layer	Choice	Why
Backend	Python 3.11 + FastAPI	async I/O for API calls; best medical/LLM library ecosystem
Validation	Pydantic v2	every boundary (LLM output, API, storage) is schema-validated
Document reading	Claude vision + forced tool call	no custom OCR pipeline; schema-valid output by construction
Drug vocabulary	RxNorm REST API (free, no key)	the industry-standard normalizer; built for fuzzy input
PDF read/write	PyMuPDF (<code>fitz</code>)	one library both rasterizes incoming PDFs and writes the export PDF
Storage	SQLite, single file	single user, zero ops; upgrade path is Postgres (see ARCHITECTURE.md)
Frontend	one static HTML file, vanilla JS	no build step, no CDNs, works offline, trivially auditable
Deployment	Windows .bat (desktop) or Docker Compose + Caddy (server)	see §9

There is deliberately **no framework magic**: no ORM, no frontend framework, no agent orchestration library. Every moving part is a plain function you can read top to bottom.

3. Repository tour

```
MedicalConcierge/
├─ Start-MedicalConcierge.bat    # double-click Windows launcher (see §9.1)
├─ docker-compose.yml           # server deployment: app + Caddy proxy (§9.2)
├─ deploy/
│   └─ Caddyfile                 # TLS-terminating reverse proxy config
│   └─ README.md                 # deployment instructions
├─ docs/                         # you are here
└─ backend/
    ├─ Dockerfile                # container image for the app (§9.2)
    ├─ requirements.txt           # runtime deps; -dev adds pytest
    ├─ static/index.html         # the entire frontend (incl. welcome splash)
    └─ app/
        ├─ main.py               # FastAPI wiring: /api router + static mount
        ├─ config.py             # Settings (pydantic-settings, reads .env)
        ├─ schemas.py            # ALL shared data models – start reading here
        ├─ api/routes.py         # every HTTP endpoint
        ├─ ingestion/            # file -> images -> extracted items
        ├─ normalization/       # names -> RxNorm codes
        ├─ agents/               # pipeline orchestration (unified document pass)
        ├─ interactions/        # screening rules + engine
        ├─ export/              # clinician PDF builder
        └─ storage/              # SQLite persistence
    └─ tests/                   # all offline; LLM + network mocked
```

Reading order for a new developer: `schemas.py` → `agents/common.py` → `ingestion/` → `normalization/` → `interactions/engine.py` → `api/routes.py` → `static/index.html`. Roughly 1,600 lines of Python total — an afternoon.

4. The data model (`app/schemas.py`)

Four models matter; everything else in the system is a function between them.

ExtractedItem — one medication/supplement mention *as read off a document*, before anyone decides what it actually is. Fields: `raw_text` (verbatim, so a human can audit the reading), `name_as_written`, `dosage`, `form`, `route`, `frequency`, `prescriber_or_source`, `date_documented`, `source_type`, `extraction_confidence` (0–1, set by the model against a rubric), and `ambiguities` (a list of plain-language notes like *"Dose digit unclear — could be 10 or 40"*).

RxNormMatch — what normalization concluded: `rxcuri` (RxNorm's concept ID), `canonical_name`, `match_score` (RxNorm's 0–100), a derived `normalization_confidence` (0–1), and `source` (`"rxnorm"` or `"local_supplement_table"`).

NormalizedRecord — the persisted unit: an `ExtractedItem` + an `RxNormMatch` + derived fields. The important logic is in `NormalizedRecord.build()`:

```
overall = extracted.extraction_confidence * normalization.normalization_confidence
needs_review = overall < review_threshold # default 0.6
```

The multiplication is deliberate: a perfectly-read name that failed to normalize, and a beautifully-normalized guess at illegible handwriting, should **both** score low. Either failure alone is disqualifying, so the combiner is a product, not an average.

Finding — one screening result (interaction, duplicate, or depletion): severity (`major` / `moderate` / `info`), category, title, the involved record IDs and display names, a plain-language `explanation`, a discussion-framed `recommendation`, an `evidence_note`, and `reading_confidence` — the *minimum* `overall_confidence` among involved records, because a finding is only as trustworthy as the shakiest reading underneath it.

5. Life of a document

The best way to understand the app is to follow one upload end to end. Say the user photographs a pill bottle and clicks **Read this document**.

5.1 Upload (`static/index.html` → `POST /api/ingest/document`)

The frontend sends a `multipart/form-data` request with just the file. There is deliberately nothing else to select: a field audit showed that user-picked type hints (medicine vs supplement, printed vs handwritten) *cause* errors on real mixed documents — a facility medication chart contains both kinds and both writing styles at once, and an upload labeled "medicine" used to skip every supplement on it. The model classifies each item's `kind` and `source_type` from the page itself. (The old `/ingest/medicine` and `/ingest/supplement` endpoints remain as aliases to the same unified pass.)

5.2 File → images (`ingestion/file_loader.py`)

`load_as_images()` returns a list of `(image_bytes, mime_type)` tuples:

- Small images (`.jpg/.jpeg/.png/.webp`) pass through byte-for-byte.
- **Oversized images are downscaled and re-encoded.** The vision API rejects individual images over ~5 MB and internally downscales anything past ~1568 px on the long side — so a 12 MP phone photo sent raw is both a "file too large" error and wasted upload. Anything beyond 1568 px or 1.5 MB is rendered down to a 1568 px long-side JPEG (quality 80) with PyMuPDF. No accuracy is lost; the API would have discarded those pixels.
- **PDFs are always rasterized** to one JPEG per page, at the configured DPI but never beyond the same 1568 px cap, with a 50-page guardrail.

The API layer above this adds a 30 MB request cap with a friendly message and converts vision-API failures into clean HTTP 502s instead of raw 500s.

Why rasterize even text-based PDFs? Real-world medical PDFs are frequently scans or faxes with no text layer. A "try the text layer, fall back to images" branch would double the code paths for zero reliability gain — and a printed page is a trivially easy image for a vision model. One uniform path.

5.3 Vision extraction (`ingestion/multimodal_extractor.py`)

Page images go to Claude alongside a system prompt that makes the model a "meticulous medical records transcriptionist." Short documents fit one request; long ones are **batched** — at most 5 pages and ~15 MB of raw image bytes per request (the API caps total request size at ~32 MB with base64 inflating by 4/3, and dense pages each cost output tokens), with extracted items merged in page order. The output budget is 16 K tokens, and if a response still hits that ceiling (`stop_reason == "max_tokens"` , meaning the tool-call JSON was cut off) the batch is **split in half and retried** — truncation must never be mistaken for "no medications found," which was a real field bug: a 5-page facility medication chart produced more item JSON than the old 4 K budget, the truncated tool input parsed as empty, and the app reported zero records. Malformed individual items are dropped with a logged warning rather than failing the whole document. Three techniques here are the heart of the ingestion design:

1. **Forced tool call.** The request defines a `record_extraction` tool with a full JSON Schema and sets `tool_choice={"type": "tool", ...}` . The model *cannot* reply with prose; the API guarantees a schema-shaped response. There is no "parse the model's JSON and hope" step anywhere. The result is then re-validated through Pydantic (`ExtractedItem`) as a second belt-and-suspenders layer.
2. **An explicit confidence rubric** in the prompt (0.9+ only for clearly printed text; 0.3–0.59 for partially legible handwriting; etc.). Without anchoring, model self-reported confidence is noise; with it, the numbers are consistent enough to drive the `needs_review` threshold.
3. **Anti-guessing instructions.** The model must extract crossed-out or ambiguous items *and say so* in `ambiguities` rather than silently resolving or omitting them, must report verbatim `raw_text` for auditing, and must never infer a drug identity the visible text doesn't support. Field-audit-driven rules: exactly ONE item per prescription (a line showing "Eliquis (apixaban)" is one item, not two); a crossed-out/ superseded dose belongs in the current item's `ambiguities` , never as a separate item; and after the first pass the model re-scans specifically for supplements/vitamins/minerals — the most commonly missed class.
4. **A verification pass** (`ENABLE_VERIFICATION_PASS` , default on): each batch gets a second look — same pages plus the list of what was already extracted, with instructions to report **ONLY** missing items. This is the recall net for the worst failure class (an entire category of items silently skipped). Best-effort: its failure never sinks an upload, and name-level de-dup guards against the model repeating known items.

5.4 Normalization (`normalization/`)

Each extracted name goes to RxNorm's `approximateTerm.json` endpoint — purpose-built for misspellings, brand names, and OCR noise — queried with `option=1` (current concepts only), so retired concepts like "INSULIN BEEF LENTE" can never win. Candidate selection is then accuracy-hardened in three steps (all from a field audit of real medication charts):

1. **Route/form-aware re-ranking**: an item extracted as an oral tablet heavily penalizes candidates whose names say Injection/Injectable/ Cartridge/Pen (and vice versa) — this is what stopped oral pantoprazole from mapping to "pantoprazole Injection" and triggering a false duplicate-acid-reducer warning.
2. **LLM adjudication for the ambiguous minority**: when the top candidates are within 15 points or a route conflict was seen, one cheap text-only call shows the model the extracted context plus the top-5 candidates and asks it to pick (or say "none"). Deterministic ranking handles the easy majority; the LLM only referees genuinely close calls.
3. **Ingredient resolution**: the winner is resolved to its TTY=IN ingredient (`/rxcui/{id}/related.json?tty=IN`). Brand and generic share an ingredient RxCUI — this is the identity the medication list and the screening engine use, so "Eliquis" and "apixaban" become one entry instead of two (and Cozaar+losartan stopped being flagged as "two different blood-pressure medicines").

So "Tylenol" — however the doctor scrawled it — becomes `rxcui=202433, ingredient=acetaminophen`. That's what makes downstream screening possible: every rule matches against canonical vocabulary, not against whatever was written.

Supplement-kind items get one extra step: if RxNorm's best score is below 50, a curated ~40-entry local synonym table (`supplement_terms.py`) is tried — RxNorm has vitamins but not most herbal products. Table matches get a fixed, honest `normalization_confidence` of 0.75 and `source="local_supplement_table"`. The production plan replaces this table with TRC NatMed Pro (licensed) behind the same interface.

If neither source matches, the record is stored anyway with confidence 0 — **flagged unnormalized, never silently dropped, never guessed**. RxNorm being unreachable degrades the same way instead of failing the upload.

5.5 Orchestration (`agents/`)

`agents/common.py:process_document()` is the whole pipeline in ~30 lines: load images → single-pass extract (both kinds) → for each item, normalize per its model-assigned kind → `NormalizedRecord.build()`. That's the entire "agent framework" — plain functions.

5.6 Storage (`storage/store.py`)

SQLite, one table. Each record's full Pydantic JSON goes into a `payload` column, with a few extracted columns (`kind`, `needs_review`, `overall_confidence`, timestamps) for filtering. Reads are `model_validate(json.loads(...))` — the schema is the source of truth, not the table shape. MVP-grade and explicitly unencrypted; ARCHITECTURE.md §5 covers what must change before real PHI on a shared machine.

5.7 Screening (`interactions/`) — runs after every change

The UI refetches `GET /api/findings` after each upload. The engine (`engine.py:evaluate()`) is a **pure function** of the record list: deterministic, no LLM, no network, instantly recomputable, trivially testable. It runs three passes:

- **Interaction rules** (`knowledge_base.py:INTERACTION_RULES`): each rule has two term-lists (e.g., ACE inhibitors × potassium). If any record matches side A and a *different* record matches side B, a Finding is emitted. Matching uses word-boundary regex against the canonical name (fallback: as-written), so the term `iron` can never match "Ironwood Herbal Blend".
- **Duplicate detection**: two different members of the same therapeutic class (two NSAIDs → classic two-doctors-not-talking), or the same ingredient appearing in multiple source documents.
- **Depletion rules**: e.g. metformin depletes B12. Note the tone switch: if the user *already* takes the depleted nutrient, the recommendation becomes "mention it so levels get checked" instead of "consider adding" — the engine checks the supplement list before recommending.

Findings sort most-severe first, then weakest-reading-confidence first within a band, so the user verifies shaky records early. Every rule in the knowledge base is a pharmacy-handout-level documented fact, and the UI/PDF label the whole feature as "a starter screen, not a complete check."

5.7b The welcome splash

`static/index.html` opens with a first-run overlay explaining what the app is, the four sources its information comes from (the user's documents via Claude vision, RxNorm, the built-in interaction list, and nothing else), how to get accurate results, the exact privacy boundary, and the not-medical-advice limits. Dismissal is remembered in `localStorage` (`mc_splash_seen`) — the app's only `localStorage` use, and appropriate since it's a UI preference rather than medical data; a storage failure just means the welcome shows again, which is harmless. The **How this works** header button reopens it without clearing the flag. Escape and backdrop clicks close it, and focus moves into the panel on open and back to the button on close.

5.8 Render

The frontend rerenders the record list (confidence badges, "check this" flags, ambiguity callouts) and the warnings panel (severity-colored cards with explanation + 💡 recommendation + confidence caveat). All rendering is ~100 lines of vanilla JS operating on the JSON the API returns.

5.9 The curated medication list (`storage/med_list.py`)

Raw ingested records are evidence; the **medication list** is the living, user-editable "what I actually take" layer built on top of them. Three tables in the same SQLite file:

- `list_items` — one row per medicine/supplement. Identity for de-duplication is the RxCUI when known, else the lowercased name, so "Tylenol" from a bottle photo and "acetaminophen" from a visit

note become one item. Items are never hard-deleted; "I no longer take this" is `status=stopped`, which preserves history and baseline comparability.

- **list_history** — append-only audit trail. Every mutation (created, updated, stopped, reactivated, or *observed* — the same medicine seen again in a new document, with any dose/frequency discrepancies called out) writes an event with a full item snapshot. Nothing here is ever updated or deleted.
- **baselines** — named full-list snapshots. The first ("Original baseline") is created automatically when the list first gains an item; the user can set a new one whenever their regimen is confirmed. `compare_to_baseline()` buckets the diff into added / stopped / changed (field-level before → after) / unchanged; an item both added *and* stopped since the baseline is treated as a net no-op.

Two integration rules keep it coherent:

1. **Ingestion may create items or log 'observed' events — never overwrite fields.** On-screen edits are the only way fields change; a new document claiming a different dose becomes an observed-with-differences history event for the user to reconcile, not a silent update.
2. **Screening and the doctor PDF run off the active list** once it has entries (falling back to raw records before then), via `item_to_record()`. Stopping a medicine on screen immediately clears its warnings; curated items carry full confidence since the user has confirmed them.

5.9 The SME panel (`app/panel/`)

Everything up to here is deterministic: extraction, normalization, and a screening engine that matches terms against a fixed table. The panel is the other half — an adversarial multi-agent review that reasons about the regimen instead of looking it up. It **augments** the rule screen rather than replacing it, and the UI keeps them visibly separate, because a reader needs to know which findings are a hard-coded pharmacy fact and which are AI deliberation.

The design is a port of the Panel Room / MHDebate protocol, with a roster tailored to medication management across uncoordinated prescribers.

The registry (`backend/data/panel_registry.json`)

29 subject-matter agents as data, not code, so the panel's composition is reviewable and changeable without touching the engine. Each carries a `stance`, `system_prompt`, `privileged_evidence`, `typically_challenges` (the adversarial graph), `concedes_when` (its falsifiability condition), `bias_watch` (its known failure mode, shown to the reader rather than hidden), and `required_round_output`.

Eight panels: Pharmacy & Pharmacology, Organ-System Risk, Prescribing Context, Supplements & Nutrition, Evidence & Adversarial, Data Integrity, Practice & Access, and Governance.

Three seats carry the structural load:

- `red_alert_fatigue` argues most flagged interactions are clinically inert. Every interaction tool ever built has failed in the same direction, and a reader warned about everything learns to ignore warnings.
- `red_undertreatment` counterweights the whole panel, which is structurally biased toward finding fault with what is present rather than with what is absent.
- `data_provenance` is specific to this app: it reasons about the *data*, not the pharmacology, and asks whether the regimen on the page is the one the person actually takes — reading confidence, missing doses, stale entries.

The `house_rules` array holds the binding commitments every agent inherits. The load-bearing one: *name specific drugs and doses, but never tell the reader to start, stop, or change anything*. MHDebate forbids naming medications at all; that line had to be redrawn here, since this app is entirely about medications. The rule became non-direction rather than non-specificity.

The eight rounds (`app/panel/engine.py`)

Round	What happens
1	Intake framing + seating. An urgency screen runs first, locally.
2	Independent review — every seated agent in parallel, blind to each other
3	Cross-examination along the registry's <code>typically_challenges</code> edges
4	Significance pruning — evidence grading; drop what is inert in practice
5	Lived-experience review — adherence reality, does this help or frighten
6	Feasibility + ethics gate — tier by cost; check for drift into directing
7	Synthesis and dissent
8	Safety veto — rewrite anything that reads as an instruction

Round 2 is parallel **by design**. Independence is the only structural defence against every seat agreeing because they share one underlying model, and the synthesis is required to say so to the reader in plain words.

Round 7's schema requires a non-empty `dissent` list. Disagreement that gets smoothed over costs the reader real information, so the output keeps it with equal prominence rather than collapsing to a tidy consensus.

Every boundary where a model supplies an agent id is normalised: invented ids are dropped at seating, self-pairs and unknown speakers are dropped at cross-examination, and attributions to agents that never spoke are stripped from the synthesis, so the UI cannot over-claim who backed a position.

Safety (`app/panel/safety.py`)

A deterministic, local screen that runs before any API call and grades rather than halts: `urgent` (emergency language — stops the panel and routes to care), `concern`, `sensitive`, `clear`. It reads the free-text note

only. A dangerous *combination* is a finding for the panel to reason about, not a reason to refuse to run.

Streaming

`POST /api/panel` streams server-sent events as each round and agent turn starts and finishes — an eight-round run takes a while, and watching the room fill in is most of what makes it legible. The completed review is persisted (one row, latest only) so it survives a reload, since re-running costs real money.

5.10 Labs, and closing the loop

Two additions that between them change what the panel can do.

Lab results

Labs ride the **same vision pass** as the medications (`labs` array on the extraction tool schema): a visit summary routinely carries both, and reading it twice would cost twice as much to learn the same thing.

`extract_records` returns an `ExtractionResult(items, labs)`; `process_document` returns a `DocumentResult`.

Values are stored as **strings, never floats**. Lab reports carry units, inequalities (`<0.01`) and qualitative results (`negative`), and silently coercing those is how a tool ends up confidently wrong about a number. The extractor is also explicitly forbidden from deciding a value is abnormal from its own knowledge — it copies the document's own H/L/CRITICAL flag or records `unknown`.

This matters because several agents were reasoning blind. Nephrology's own prompt said *"I usually cannot see kidney function, and here is what I am assuming."* With labs on file it analyses a number instead. When there are no labs, `regimen_brief` says so explicitly and instructs agents to state their assumptions rather than reason as though they knew.

Tracked observations (`app/panel/observations.py`)

The panel's most valuable sentences are the ones naming what would settle a disagreement — `what_would_settle_it` per dissent, plus `discriminating_observations`. On their own they are a to-do list the reader has to hold in their head. `derive()` turns them into tracked observations, dissent-sourced first (those settle an argument rather than merely informing one), de-duplicated on a punctuation-insensitive key so a re-run does not ask the same question twice.

The person answers them (`POST /api/observations/{id}/answer`) or marks them not applicable. On the next run, answered observations enter the brief as **established fact**, with agents instructed to concede explicitly where an answer meets their own `concedes_when` condition — the field that was previously decorative. Round 7 gained a `resolved` output so convergence is visible: what was settled, by which answer, and which expert changed its mind.

One ordering constraint worth knowing: `save_panel_review` deliberately does **not** clear observations, only `clear_all` does. Answered observations are the input that makes the next run converge, so wiping them on each run would silently break the loop while everything still appeared to work.

5.11 The assistant (`app/assistant/`)

The conversational layer, and the thing that turns a tool you operate into something you talk to. Two properties separate it from a general medical chatbot, and both are structural rather than stylistic.

It answers from the record. `retrieval.build_digest()` assembles the med list, labs, change history, open observations, rule findings and the latest panel review into one block, every row prefixed with its id. Answers cite those ids, and `engine._parse()` discards any citation or action whose id is not actually in the digest — so the UI can never show a reference to something invented. "Your documents don't say" is a first-class answer, and usually a useful one: a blank where an indication should be is itself worth taking to a prescriber.

No vector store, deliberately. One person's record is small enough to read whole, which removes an entire category of infrastructure and its retrieval-miss failure mode. History is the only unbounded table and is bounded at 60 recent events, with the total reported so the assistant knows what it is not seeing.

It never decides anything. Three layers, in order of reliability:

1. The deterministic urgency screen (`app/panel/safety.py`) runs on every turn before any API call — the same gate the panel uses.
2. The response schema gives clinical advice nowhere to live: anything the assistant is not allowed to decide goes in `questions_for_clinician`, phrased for the person to say out loud.
3. `guard.py` is a backstop that flags directive language in the output. It is deliberately narrow — it matches only the *assistant itself* recommending ("I'd suggest reducing...", "you should stop..."), not reporting ("your cardiologist told you to stop...") or recording ("you stopped it in March"). A naive scan for "stop taking" fires on both of those, which are core to what this assistant does, and a guard that cries wolf gets switched off. When it fires the answer is shown with a visible notice rather than silently rewritten, because quietly editing a medical answer is its own failure mode.

Proposed actions

"Just got back from cardiology, they upped the warfarin to 7.5 and said stop the fish oil" produces two `ProposedAction`s — and changes nothing. Each is a confirm button; `POST /api/chat/actions/{id}/apply` is the only thing that writes. A misheard sentence silently editing a medication list is precisely the failure this app exists to prevent, so the human stays in the loop by construction. Actions can also answer the panel's open observations, which closes the loop from §5.10 conversationally.

6. The API surface (`app/api/routes.py`)

Every route below is served at **both** `/api/...` and `/api/v1/...`. The bundled web UI uses the unversioned path; a native client should pin to `/api/v1` so a future breaking change can ship as `/api/v2` without stranding installed apps. See [APP_ROADMAP.md](#).

Endpoint	What it does
GET /api/health	{ok, anthropic_key_configured} — drives the UI's "Setup needed" banner
POST /api/ingest/document	multipart upload → unified pipeline (medicines + supplements, model-classified) → stored records returned
POST /api/ingest/medicine / .../supplement	back-compat aliases for the same unified pass
GET /api/records?kind=	all stored records, newest first
GET /api/findings	screening results, recomputed on the fly (active list items when the list is non-empty)
GET /api/list	curated list items + baseline summaries
POST /api/list/items	manual add (normalized through RxNorm/local table)
PATCH /api/list/items/{id}	on-screen edit: dose, frequency, notes, status
GET /api/list/history?item_id=	append-only audit trail
POST /api/baselines	snapshot the current list under a name
GET /api/list/compare/{baseline_id}	added / stopped / changed since that baseline
GET /api/export?format=json\ csv\ pdf	full export; JSON includes the list, baselines, and history; PDF is the clinician summary
POST /api/chat	one grounded turn; returns answer, citations, proposed actions, clinician questions
GET /api/chat / DELETE /api/chat	conversation history; clearing it leaves records untouched
POST /api/chat/actions/{id}/apply	apply one confirmed record change
GET /api/labs	lab values read off uploaded documents
GET /api/observations	what the panel said would settle its open questions, and what has been answered
POST /api/observations/{id}/answer	record what the person found out; the next run reads it as fact
POST /api/observations/{id}/dismiss	not applicable, without pretending it was answered
GET /api/panel/roster	the 29 experts, their biases and adversarial edges — the panel is inspectable, not a black box

Endpoint	What it does
POST /api/panel	runs the eight-round review over the current list, streaming SSE progress; last event carries the review
GET /api/panel/latest	the most recent completed review, so it survives a reload
GET /api/panel/pdf	the panel's questions and unresolved disagreements, to hand over at an appointment
POST /api/reset	start over: builds a full-archive PDF (list incl. stopped items, baselines, complete history, findings, raw records), wipes both stores only after the bytes exist, and returns the PDF as the response — a generation failure aborts with nothing deleted
GET /	serves static/index.html

There is no auth: the server binds to `127.0.0.1` in desktop mode, and in Docker mode is reachable only through the reverse proxy on a LAN/VPN. Anything beyond that (passkeys/WebAuthn) is future work per ARCHITECTURE.md.

7. Exports (`app/export/pdf_report.py`)

CSV and JSON are straightforward serializations in `routes.py`. The PDF is the interesting one: it's built with PyMuPDF's low-level text API through a small `_writer` class that owns a y-cursor, measures text with `fitz.get_text_length()` for word-wrapping, and page-breaks automatically.

The layout order is intentional: **Potential Interactions & Recommendations come first** (a clinician should see warnings before the med list), then Medications, then Supplements, alphabetically. Low-confidence entries carry a red `** PLEASE CONFIRM **` marker with their ambiguity notes, and each finding block shows severity, involved records, mechanism, suggested next step, and the confidence caveat. Every page footer restates "not medical advice."

8. Configuration (`app/config.py`)

`pydantic-settings` reads environment variables and `backend/.env` (written by the Windows launcher's first-run prompt, or created by hand — one `ANTHROPIC_API_KEY=...` line is enough; everything else has defaults):

Variable	Default	Meaning
<code>ANTHROPIC_API_KEY</code>	<i>(empty)</i>	the one required secret; empty → UI shows setup banner
<code>EXTRACTION_MODEL</code>	<code>claude-sonnet-5</code>	vision model for extraction
<code>ASSISTANT_MODEL</code>	<code>claude-sonnet-5</code>	the conversational layer; short turns, many per day
<code>ENABLE_PANEL</code>	<code>true</code>	turns the SME panel off entirely
<code>PANEL_MODEL</code>	<code>claude-sonnet-5</code>	individual agent turns (short, many, parallel)
<code>PANEL_SYNTHESIS_MODEL</code>	<code>claude-opus-5</code>	synthesis and safety veto, which reason over the whole transcript
<code>PANEL_MAX_SEATED</code>	<code>8</code>	ceiling on experts seated per run — the main cost lever
<code>PANEL_CONCURRENCY</code>	<code>6</code>	parallel in-flight model calls
<code>RXNORM_BASE_URL</code>	<code>https://rxnav.nlm.nih.gov/REST</code>	swappable for tests/mirrors
<code>DB_PATH</code>	<code>./medconciierge.sqlite3</code>	Docker sets <code>/data/medconciierge.sqlite3</code>
<code>REVIEW_CONFIDENCE_THRESHOLD</code>	<code>0.6</code>	below this, records are flagged <code>needs_review</code>
<code>PDF_RENDER_DPI</code>	<code>200</code>	rasterization resolution for ingested PDFs (output is still capped at 1568 px long-side for the vision API)

9. How the app is run — two deployment modes

9.1 Desktop mode (`start-MedicalConciierge.bat`)

For the primary user — a non-technical patient on Windows — Docker is too much to ask. The launcher is a plain batch file that: finds Python (with plain-English install guidance if missing) → creates `.venv` on first run → installs `requirements.txt` (re-running only when the file changes, tracked by comparing a copied marker) → prompts for the API key once and writes `backend\.env` → starts `uvicorn` bound to `127.0.0.1:8000` → opens the browser. Closing the console window stops the server; the SQLite file persists. No command line knowledge needed.

9.2 Server mode: how Docker is used

For an always-on home server/NAS/VPS, the repo ships a two-container Docker Compose stack. **Why containers here?** Reproducibility (pinned Python and dependencies independent of the host), painless updates (`git pull && docker compose build && up -d` with data untouched), process supervision (`restart: unless-stopped`), and network isolation (the app container is never directly exposed).

The image (`backend/Dockerfile`)

```
FROM python:3.11-slim          # small official base
RUN groupadd -r medconcierge && useradd -r -g medconcierge medconcierge
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt    # own layer: cached
COPY app ./app                                # unless requirements change
COPY static ./static
RUN mkdir -p /data && chown -R medconcierge:medconcierge /data /app
VOLUME ["/data"]                      # persistent data lives here
ENV DB_PATH=/data/medconcierge.sqlite3
EXPOSE 8000
USER medconcierge                    # never run as root
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Things worth teaching from these few lines:

- **Layer-cache ordering:** `requirements.txt` is copied and installed *before* the application code, so day-to-day code changes rebuild in seconds — the dependency layer only rebuilds when requirements change. (`.dockerignore` keeps `.venv`, tests, `.env`, and caches out of the build context, which also prevents the real API key from ever being baked into an image.)
- **Non-root user:** a container escape or app compromise lands in an unprivileged account.
- **0.0.0.0 inside the container is safe** because of the network design below: the port is exposed to the internal Docker network only, never `ports`-published to the host.
- **State separation:** the image is disposable; everything worth keeping is under `/data`, a declared volume.

The stack (`docker-compose.yml`)

```
services:
  app:                                # FastAPI, built from backend/Dockerfile
    env_file: ./backend/.env
    volumes: [medconciierge-data:/data]
    expose: ["8000"]                 # internal network only — no host port
    healthcheck:                     # GET /api/records must succeed
    networks: [internal]
  proxy:                              # Caddy, the ONLY container with host ports
    image: caddy:2-alpine
    depends_on: {app: {condition: service_healthy}}
    ports: ["443:443", "80:80"]
    volumes: [./deploy/Caddyfile:/etc/caddy/Caddyfile:ro, caddy-data:/data, ...]
    networks: [internal]
```

The two containers share a private `internal` network. The proxy resolves the app by its service name — that's the `reverse_proxy app:8000` line in the Caddyfile; Docker's embedded DNS maps `app` to the container's internal IP. The `depends_on ... service_healthy` gate means Caddy won't start routing until the app's healthcheck (an actual API call, not just "process exists") passes.

Three named volumes survive rebuilds: `medconciierge-data` (the SQLite database — the only thing that truly matters; the backup command in `deploy/README.md` tars exactly this), plus Caddy's cert/config state.

The proxy (`deploy/Caddyfile`)

Caddy terminates TLS and adds security headers. Because this system is LAN/VPN-only *by design* (no public domain), it uses `tls internal` — Caddy mints its own CA and issues itself a cert; the user trusts that root cert once per device (or skips Caddy entirely and uses `tailscale serve`, which brings its own certs — both paths are documented in `deploy/README.md`). Port 80 exists only to redirect to 443.

The deliberate security posture, from ARCHITECTURE.md §7: **never expose this to the public internet**. Remote/phone access goes through a personal VPN (Tailscale/WireGuard). The only outbound traffic is the Claude API (document images) and RxNorm (drug names only) — and that scope is documented to the user rather than pretending the system is air-gapped.

Day-2 operations

```
docker compose build && docker compose up -d    # deploy / update
docker compose logs -f app                       # watch the app
docker compose ps                                # health status
# backup (see deploy/README.md):
docker run --rm -v medconciierge-data:/data -v "$PWD":/backup alpine \
  tar czf /backup/medconciierge-data-$(date +%F).tar.gz -C /data .
```

10. Testing (`backend/tests/`)

```
cd backend && pip install -r requirements-dev.txt && pytest
```

All tests run **offline with no API key** — that's a design requirement, not an accident. The seams that make it possible:

- The Claude call is one function (`extract_records`); agent tests monkeypatch it at its import site in `agents/common.py` and feed hand-built `ExtractedItems`.
- The RxNorm client's HTTP layer is `httpx.AsyncClient.get` ; tests monkeypatch that with canned JSON responses.
- The screening engine and PDF builder are pure functions — tests feed records in and assert on findings out (or extract text back out of the generated PDF with PyMuPDF and assert on it).

What the suite covers: confidence math and thresholds, RxNorm candidate selection and the empty/failure paths, the supplement fallback decision, both agents end-to-end (with mocks), ten screening-engine behaviors (severity, categories, duplicate detection, word-boundary matching, depletion tone switching, sort order, confidence propagation), and PDF content/pagination. What it deliberately can't cover: real Claude reading accuracy and live RxNorm behavior — those need a manual smoke test with a real key.

11. Extending the system

Add an interaction rule — append an `InteractionRule` to `INTERACTION_RULES` in `knowledge_base.py` (or a `DepletionRule` , or a class in `DUPLICATE_CLASSES`). Terms are matched with word boundaries against canonical names; add a test in `test_interaction_engine.py` . No other file changes — the engine, API, UI, and PDF all pick it up.

Swap in a real interaction database (openFDA, NatMed Pro) — implement a provider that yields the same rule/finding shapes and merge its output in `engine.evaluate()` . The UI and PDF render `Finding s`; they don't care where rules came from. Keep the curated list as the offline fallback.

Add a document type (e.g. lab results) — new extractor prompt + schema in `ingestion/` , a normalization target (LOINC for labs), and a new `RecordKind` . `agents/common.py` shows the pattern to copy.

The bigger roadmap — diagnosis gap analysis, supplement evidence grading, FHIR export, encryption at rest, WebAuthn — is sequenced with rationale in `ARCHITECTURE.md` §8.

12. Privacy model (know this before touching anything)

Data that leaves the machine, exhaustively: (1) document images/text → the Claude API for extraction; (2) drug/supplement *names only* → RxNorm. Nothing else. No analytics, no telemetry, no cloud storage. All records live in one local SQLite file the user can copy, back up, or delete. When you add features, preserve this property or document the change loudly — it is the product's core promise.

Appendix A — Glossary

Term	Meaning
RxNorm	NLM's standard vocabulary of drugs; the universal translator between "Tylenol", "APAP", and "acetaminophen"
RxCUI	RxNorm Concept Unique Identifier — the stable ID for one drug concept
Normalization	mapping free-text names onto standard codes so different spellings unify
Forced tool call	an Anthropic API mode where the model must respond by "calling" a developer-defined, JSON-Schema-validated function — structured output by construction
Rasterization	rendering a PDF page into a bitmap image (here: 200 DPI PNG)
<code>needs_review</code>	record flag set when <code>overall_confidence < 0.6</code> ; surfaces as "check this" in the UI
Finding	one screening result: interaction, duplicate therapy, or nutrient depletion
Reverse proxy	the Caddy container: terminates TLS, adds headers, forwards to the app over the internal Docker network
Named volume	Docker-managed persistent storage that outlives containers; where the SQLite DB lives in server mode
<code>tls internal</code>	Caddy's self-managed private CA — right for LAN/VPN services that will never have a public domain

Appendix B — Quick-start for development

```
git clone <repo> && cd MedicalConcierge/backend
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt
echo "ANTHROPIC_API_KEY=sk-ant-..." > .env # only needed to test real ingestion
pytest # 22 tests, all offline
uvicorn app.main:app --reload # http://localhost:8000
```

To regenerate this guide's PDF: `python docs/build_dev_guide.py` (requires `pip install markdown playwright` and a Chromium — see the script's docstring).